

Dictionaries

 <https://github.com/learn-co-curriculum/python-p3-dictionaries>  <https://github.com/learn-co-curriculum/python-p3-dictionaries/issues/new>

Learning Goals

- Utilize Python's `dict` data type to accomplish several common programming tasks.
- Execute and test Python code using the Python shell.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **Mutable**: an object that can be changed.
- **Immutable**: an object that cannot be changed. (*Many immutable objects appear mutable because programmers reuse their names for new objects.*)
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

Mappings are the next data structure we're going to look into. A mapping object maps an immutable value to an arbitrary value. Where there are many types of sequences in Python, there is only one type of mapping: the `dict`, or dictionary.

In a Python dictionary, an immutable *key* is mapped to an arbitrary *value*. The result is a data type nearly identical to JSON:

```
my_dict = {  
    "key 1": "value 1",  
    2.0: "value 2",  
}
```

Keys can be composed of any immutable data type. The immutable types that we have discussed so far are:

- `int`
- `float`
- `str`
- `bool`
- `tuple`
- `range`

While there are no explicit guidelines on which types of values to use for `dict` keys, it is best to use descriptive strings unless the task you are trying to accomplish requires you to use another data type.

When Are Dictionaries Used?

Dictionaries are best used when there is a fixed name or value that we want to associate with a piece of data. An excellent analogy for a Python dictionary is...a dictionary!

We've introduced some new vocabulary in this lesson, so let's organize it into a dictionary:

```
key_vocab = {  
    "Mapping": "a data structure in which data is stored in random order \\  
        and accessed using immutable keys.",  
    "Arbitrary": "describes data whose value is unimportant to the data type \\  
        or structure in which it is contained."
```

```
}

key_vocab["Mapping"]
# "a data structure in which data is stored in random order and accessed using immutable keys."
key_vocab["Arbitrary"]
# "describes data whose value is unimportant to the data type or structure in which it is contained."
```

Definitions are data that we need to access when we hear certain words. Using those words as keys makes it easy for us to access their definitions when we need to, without parsing through every other piece of data in the data structure. It is important that the keys themselves are of an immutable type (`str`) so definitions are always connected to the correct word.

Dictionaries can also be used to store many types of data that would describe the dictionary object itself. This is the use case most similar to JSON and allows for easy communication between Python and JavaScript. We'll look more into this in Phase 4.

```
software_engineer = {
    "languages": ["JavaScript", "Python"],
    "certifications": ["Flatiron School Certificate of Completion"],
    "experience": "3 months and counting!",
}
```

We can also use dictionaries as a replacement for switch/case statements, as we saw in the lesson on conditional statements. We call this approach **dictionary mapping**.

```
size_to_ounce_map = {
    "tall": 12,
    "grande": 16,
    "venti": 20,
}
```

Now if we wanted to train a machine to pour the correct amount of coffee for each cup size, we could wrap this dictionary in a function where we enter our cup size and receive a volume in return:

```
def pour_coffee(size):  
    size_to_ounce_map = {  
        "tall": 12,  
        "grande": 16,  
        "venti": 20,  
    }  
    return size_to_ounce_map[size]
```

NOTE: If you choose to use dictionary mapping instead of an `if/elif/else` statement, make sure you're using it to write shorter, clearer code.

Dictionary Methods

Getting Data

The most important dictionary method is `dict.get()`. Open up the Python shell and enter our code from the `pour_coffee()` function above:

```
def pour_coffee(size):  
    size_to_ounce_map = {  
        "tall": 12,  
        "grande": 16,  
        "venti": 20,  
    }  
    return size_to_ounce_map[size]
```

In the above example, we're using simple bracket notation - `[]` - to access key value pairs within our `size_to_ounce_map`.

Now let's test some different values for size:

```
pour_coffee("tall")  
# 12
```

```
pour_coffee("grande")
# 16
pour_coffee("venti")
# 20
pour_coffee("jumbo")
# KeyError: 'jumbo'
```

This error helps us, but it doesn't help our poor coffee machine pour coffee. `dict.get()` gives us some tools to avoid `KeyError` s. Let's change our `return` statement to use `dict.get()` :

```
def ...
    return size_to_ounce_map.get(size)

pour_coffee("jumbo")
# None
```

We've managed to avoid a `KeyError` , but `None` might not be very helpful either. Let's make one last change:

```
def ...
    return size_to_ounce_map.get(size, "Please enter a valid cup size.")

pour_coffee("jumbo")
# 'Please enter a valid cup size.'
```

`dict.get()` takes two arguments. The first is required- this is the key that you're searching for in your dictionary. The second optional argument is a default value for your search (if you don't enter a second argument, Python uses `None` as a default value). `dict.get()` is the best way to retrieve values from a Python dictionary if you need to avoid `KeyError` exceptions.

Setting Data

In addition to getting data from a dictionary, we can add new data to a dictionary and update existing data within a dictionary.

As with getting data, there are a variety of ways to set data.

The simplest way is to just use bracket notation - `[]` - and the assignment operator, much like JavaScript:

```
my_dict = {
    "key_one": "value one",
    "key_two": "value two",
}

# update an existing key-value pair:
my_dict["key_one"] = "I've changed!"

# set a new key-value pair:
my_dict["key_three"] = "value three"

print(my_dict)
# prints { "key_one": "I've changed!", "key_two": "value two", "key_three": "value three"}
```

However, Python also offers us a powerful method - the `update` method - that can be used to set large amounts of data within an dictionary.

The `update` method can be used to add multiple new key-value pairs at the same time, update multiple fields at the same time, or do both:

```
my_dict = {
    "key_one": "value one",
    "key_two": "value two",
}

# update multiple fields:
my_dict.update({"key_one": "new value one", "key_two": "new value two"})

# add multiple fields:
my_dict.update({"key_three": "value three", "key_four": "value four"})

# add and update fields simultaneously:
my_dict.update({"key_three": "new value three", "key_four": "new value four", "key_five": "value five"})
```

```
print(my_dict)
```

In this example, we passed a dictionary as the argument to `update`. However, you can also pass in tuples and arrays as arguments or use assignment operation:

```
my_dict = {  
    "key_one": "value one"  
}
```

```
# passing in an array of arrays
```

```
my_dict.update([["key_one", "new value one"], ["key_two", "value two"]])
```

```
# passing in an array of tuples
```

```
my_dict.update([("key_two", "new value two"), ("key_three", "value three")])
```

```
# passing in a tuple of arrays
```

```
my_dict.update(("key_three", "new value three"), ["key_four", "value four"])
```

```
# passing in a tuple of tuples
```

```
my_dict.update(("key_four", "new value four"), ("key_five", "value five"))
```

```
# using assignment operation
```

```
my_dict.update(key_five="new value five", key_six="value six")
```

```
print(my_dict)
```

```
# prints {'key_one': 'new value one', 'key_two': 'new value two', 'key_three': 'new value three', 'key_four': 'new val
```

Iterating Over Dictionaries

The next most commonly used `dict` method is `dict.items()`, which returns an object that can be treated as a list of tuples of key/value pairs.

That's a dense definition, so let's unpack it a bit:

```
my_dict = {  
    "a": 1,  
    "b": 2,  
    "c": 3,  
    "d": 4,  
}
```

```
[key for key in my_dict]  
# ['a', 'b', 'c', 'd']  
[my_dict[key] for key in my_dict]  
# [1, 2, 3, 4]
```

We can iterate through a dictionary similarly to how we would a sequence, using either a loop or a comprehension. When we iterate through a dictionary, though, we can only access elements using keys. The object returned by `dict.items()` makes keys and values accessible independently of one another:

```
my_dict = {  
    "a": 1,  
    "b": 2,  
    "c": 3,  
    "d": 4,  
}
```

```
[item for item in my_dict.items()]  
# [('a', 1), ('b', 2), ('c', 3), ('d', 4)]  
[key for key, value in my_dict.items()]  
# ['a', 'b', 'c', 'd']
```



```
[value for key, value in my_dict.items()]  
# [1, 2, 3, 4]
```

`dict` objects are very useful when accessing one key/value pair at a time, but `dict.items()` is an important tool in writing elegant loops and list comprehensions.

There are many more `dict` methods, but these are the most common that you will see in your software career. A full list of `dict` methods can be found in the [Python dict documentation](https://docs.python.org/3/tutorial/datastructures.html#dictionaries) [↗\(https://docs.python.org/3/tutorial/datastructures.html#dictionaries\)](https://docs.python.org/3/tutorial/datastructures.html#dictionaries).

Resources

- [Python Mapping Types](https://docs.python.org/3/library/stdtypes.html#typesmapping) [↗\(https://docs.python.org/3/library/stdtypes.html#typesmapping\)](https://docs.python.org/3/library/stdtypes.html#typesmapping)
- [Python Dictionaries](https://docs.python.org/3/tutorial/datastructures.html#dictionaries) [↗\(https://docs.python.org/3/tutorial/datastructures.html#dictionaries\)](https://docs.python.org/3/tutorial/datastructures.html#dictionaries)
- [Stack Overflow: Using a dictionary as a switch statement in Python](https://stackoverflow.com/questions/21962763/using-a-dictionary-as-a-switch-statement-in-python) [↗\(https://stackoverflow.com/questions/21962763/using-a-dictionary-as-a-switch-statement-in-python\)](https://stackoverflow.com/questions/21962763/using-a-dictionary-as-a-switch-statement-in-python)
- [Python Dictionary Methods](https://www.geeksforgeeks.org/python-dictionary-methods/) [↗\(https://www.geeksforgeeks.org/python-dictionary-methods/\)](https://www.geeksforgeeks.org/python-dictionary-methods/)